Projet POO ING1

Conception et développement d'une application informatique

Christophe Meynard, Théo Szanto, Maxim Spur, Alexane Nghien

21/04 - 05/05

Première journée

1) **Présentation** du projet et des consignes
2) **Analyse** du projet, modélisation
3) **Validation** des choix du groupe avec un.e enseignant.e

Les grandes lignes du projet

Vous allez créer un jeu inspiré de "**Cartographers**" (*jeu de société*).

Certaines fonctionnalités minimales sont demandées

L'utilisateur de votre application devra pouvoir :

- créer des motifs en posant des polyominos sur une grille
- marquer des points pour gagner ou perdre la partie

Objectifs pédagogiques

- Mettre en œuvre plusieurs matières vues cette année :
  - l’algorithmie ;
  - les outils de gestion de projet ;
  - l’analyse informatique ;
  - la programmation orientée objet en Python.

Consignes

- Vous devrez :
  - Formaliser une analyse du sujet avec les outils UML ;
  - Écrire des algorithmes ;
  - Mettre en œuvre les concepts de l’orienté objet ;
  - Créer, charger, manipuler, visualiser des données ;
  - Produire une documentation de votre code ;
  - Rédiger un rapport d’analyse.

Encadrement

- Vous serez dans les salles : L207-208 / C310
- Une séance sera en autonomie (planning sur la slide suivante)
- 4 enseignant.e.s : **Christophe Meynard, Théo Szanto, Maxim Spur, Alexane Nghien**

Organisation générale

20 groupes de 3 étudiant.e.s

1 groupe de 2 étudiant.e.s

Créneau	C310	L207-208
Mardi 21/04 matin	Alexane Nghien, Maxim Spur	Christophe Meynard, Théo Szanto
Mardi 21/04 aprem	Alexane Nghien, Maxim Spur	Christophe Meynard, Théo Szanto
Vendredi 24/04 matin	Alexane Nghien, Maxim Spur	Christophe Meynard, Théo Szanto
Vendredi 24/04 aprem	Maxim Spur	Christophe Meynard, Théo Szanto
Lundi 27/04 matin	Alexane Nghien	Christophe Meynard, Théo Szanto
Lundi 27/04 aprem	Alexane Nghien	Christophe Meynard, Théo Szanto
Mardi 28/04 matin	Maxim Spur	Christophe Ménard
Mardi 28/04 aprem	Autonomie	Autonomie
Lundi 04/05 matin	Maxim Spur	Christophe Ménard
Lundi 04/05 aprem	Maxim Spur, Théo Szanto	Christophe Ménard
Mardi 05/05 matin	Alexane Nghien, Maxim Spur	Christophe Ménard, Théo Szanto
Mardi 05/05 aprem	Alexane Nghien, Maxim Spur	Christophe Ménard, Théo Szanto

Livrables

1. **Code source** sur GitHub (slide 10)
2. **Rapport d’analyse**, 5 pages max (slide 11)

Deadlines

1. Dernier commit sur votre dépôt GitHub : **4 mai 23h59**
2. Rapport d’analyse à envoyer avant le **5 mai 18h00**
3. « Présentation » lors de la journée du **mardi 5 mai**

Livrable 1 : Code source

- À déposer dans votre dépôt **GitHub**
- Le code doit être **documenté**…
- …et accompagné d'un **manuel d'utilisation** dans un fichier `README.md`, à la racine de votre dépôt
- Toutes les informations devront être disponibles dans votre dépôt GitHub
- Créez votre dépôt en faisant un fork dans l'organisation : https://github.com/ENSG-POO-2026-ING1/Projet-ING1

Livrable 2 : Rapport d'analyse

- Le rapport d’analyse permet de **formaliser** et d’**expliquer** votre compréhension du sujet et sa prise en main
- Vous illustrez votre modélisation à l’aide de **diagrammes UML**
- Vous devez réaliser au moins **3 diagrammes** différents

Livrable 2 : Diagrammes

Vous devez réaliser 3 diagrammes :

- un diagramme de **classe** ;
- un diagramme de **cas d'utilisation** ;
- un diagramme **au choix**, différents des précédents.

Présentation (mardi 05/05 après-midi)

Ce n’est PAS une soutenance

Nous passerons auprès des groupes pour :

- Observer l’application tourner sur vos machines ;
- Vous poser une série de questions ;
- Évaluer l’aspect fonctionnel.

Matériel de départ

- Cette présentation
- Accès à l’organisation GitHub “ENSG-POO-2026-ING1” pour **faire un fork** du dépôt

Évaluation (à titre indicatif)

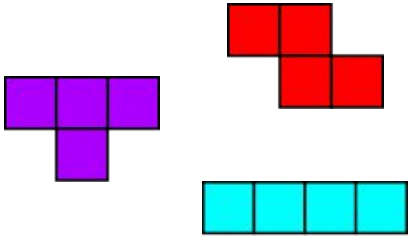
Livrable	Critères	Barème
Code source	● INTERDICTION d'utiliser Pygame (et tout autre librairie de jeu) ● Utilisation de GitHub ● POO : classes, héritage, instanciations des objets, attributs, surcharge… ● Architecture du code (p. ex. : séparation code interface / code métier) ● Clarté des commentaires, respect des conventions de codage ● Présence de tests et documentation (`README.md`, commentaires dans le code) ● Pour la présentation : l'application se lance bien, la partie se déroule correctement, tourne au moins pour 1 partie sans crash	12

Évaluation (à titre indicatif)

Livrable	Critères	Barème
Rapport d'analyse	● Reformulation et adaptation du sujet ● Modélisation de 3 diagrammes UML (classe & cas d'utilisation et 1 au choix) ● Explication des fonctionnalités, des algorithmes ● Forme du rapport (présentation, propos clairs, orthographe…)	8

Détails du projet

Règles du jeu



On dispose d’une **grille rectangulaire** de N*M cases

À chaque tour, on pioche un **polyomino** (ex : pièce Tetris) avec une certaine **couleur**

On doit positionner ce polyomino **sur la grille** (sans chevaucher une case pleine)

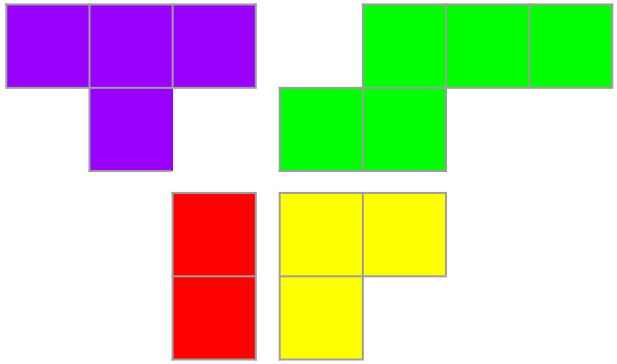
Au bout d’un nombre défini de tours, la partie se termine et on **compte les points**

Le jeu dispose de cartes de score définissant des **motifs** qui rapportent des points

Exemple :

- Une colonne de 5 cases bleues rapporte 7 points
- Chaque case rouge adjacente à une case verte rapporte 1 point…

Exemple 1

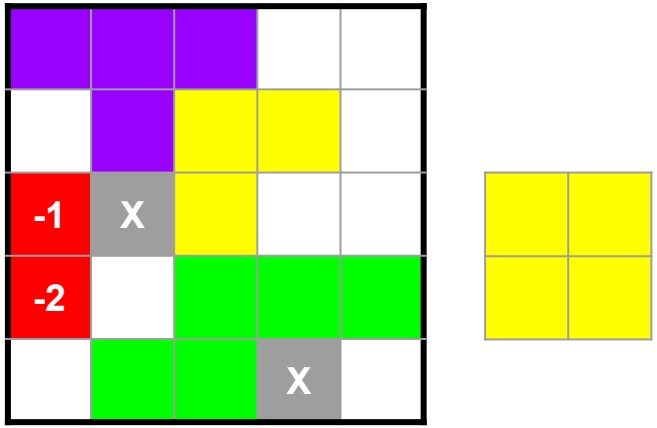


4 polyominos placés sur une **grille 5x5**

2 cases **bloquées** (en gris avec une croix)

1 polyomino “**malus**” (en rouge avec des nombres négatifs)

1 autre polyomino (carré 2x2 jaune) qui **ne rentre plus** dans la grille



Exemple 2

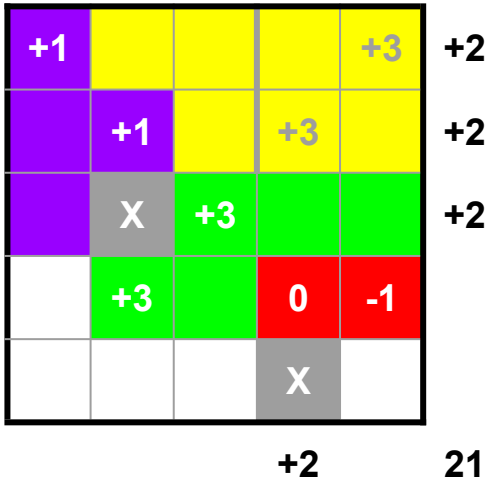
Une **autre configuration** avec les mêmes polyominos (ici avec rotation autorisée)

Les deux polyominos jaunes sont **adjacents** (ici visuellement séparés par une bordure plus épaisse)

Le polyomino malus a été **partiellement contré**

Exemple de **scores** :

- -1 par case vide adjacente à une case malus
- +2 par colonne ou ligne entièrement pleine
- +3 par case dans la diagonale du plus grand carré
- +1 par case violette adjacente à une case jaune



Fonctionnalités minimales attendues

- Grille environ **10x10** sur laquelle le joueur peut positionner librement ses polyominos
- Certaines cases sont aléatoirement **bloquées** en début de partie (impossible de poser un polyomino dessus)
- Minimum **4 façons différentes** (cartes de score avec motif différent) de gagner des points par partie
- Minimum **5 tours** par partie
- Minimum **5 formes** différentes et **3 couleurs** différentes
- Gérer le cas où le joueur **ne peut pas poser** de polyomino car pas de place

Fonctionnalités supplémentaires

Voici quelques exemples de fonctionnalités en plus :

- Des **polyominos malus** sont aléatoirement à poser (font perdre des points)
- **Rotation et réflexion** (miroir) des polyominos
- **Choix** entre plusieurs polyominos et/ou couleurs, dont certains ont des **bonus de score**
- **Tableau des scores** (leaderboard)

Vous DEVEZ ajouter au moins 1 fonctionnalité libre, à faire valider à la 1ère séance. Sentez-vous libre d'ajouter des mécaniques et/ou du contenu supplémentaire. Cela sera pris en compte positivement lors de la notation.

Concernant l’utilisation de l’IA

Vous avez le droit d’utiliser l’IA (LLM, assistant de code…) à une condition : **COMPRENDRE**.

Si vous comprenez ce que l’outil vous propose, ok.

Si vous vous contentez de copier/coller sans réfléchir, non.

Pourquoi ? Car le but de ce projet, c’est d’apprendre !

Vous devrez être en mesure de répondre à n’importe quelle question concernant le code. La réponse “*je ne sais pas, c’est l’IA qui a fait ça*” ne sera pas tolérée !

Concernant l’interface :

Interdiction d’utiliser Pygame, Python Arcade (et toutes les librairies de création de jeu python)

Recommandations :

- **PyQT (avec QT Designer) ou PySide**
- Terminal (ex : PyPlayScii https://github.com/k2sebeom/pyplayscii)
- *Si vous choisissez une autre solution, vous devez impérativement la faire valider auprès d’un enseignant !*

À faire maintenant pour vous lancer

D’ABORD, réfléchir :

- Se mettre d’accord sur les détails du projet et faire valider par un enseignant
- Modélisation UML (quelles structures de données, comment les éléments s’instancient/interagissent entre eux ?)
- Répartir les tâches entre les membres du groupe (qui fait quoi ?)

ENSUITE, coder :

- Implémenter l’interface graphique (qui découlera de votre modélisation)
- Implémenter les classes / éléments / méthodes etc

Groupe 1		Groupe 12	
Groupe 2		Groupe 13	
Groupe 3		Groupe 14	
Groupe 4		Groupe 15	
Groupe 5		Groupe 16	
Groupe 6		Groupe 17	
Groupe 7		Groupe 18	
Groupe 8		Groupe 19	
Groupe 9		Groupe 20	
Groupe 10		Groupe 21	
Groupe 11			